Introduction to Odometry: Moving and Controlling the Motors

TH Köln – Advanced Robotics, Laboratory – Lab report,

[Abhishek Chandorkar]

[Chinmay Vaity]

Lecturer: Prof. Elena Algorri – 08 November 2026

Abstract

We built a two-wheeled robot with two Parallax Feedback 360 servos and an Arduino Nano 33 BLE and used the servos' built-in position feedback for odometry. Starting from a timed open-loop sketch, we developed one firmware that moves by distance and angle, keeps both wheels synchronised with a PI controller, and tracks the robot pose (x, y, θ) by dead reckoning. A 2 m out-and-back run (Project 3) returns to the start within about 1 cm to 2 cm. Thirty 40 cm runs (Project 4) gave a covariance matrix with standard deviations of 1.5 mm along the line and 2.6 mm sideways, and thirty right and thirty left 90° turns gave angular standard deviations of 1.35° and 1.18°. For Project 5 (bow-tie lap followed by square lap), the robot plans every turn and every side from its odometric pose, learns its stopping point per turn size, and uses separate effective wheelbases for right and left turns. Our best two-lap runs ended within 0.5 cm to 1.5 cm of the origin, against a tolerance of 5 cm. A Monte Carlo model fitted to our logs shows that the remaining error is mostly wheel slip that the encoders cannot observe.

1. Task and overview

The lab handout defines four graded projects and a report (9 points in total). The table summarises them together with the firmware command we use for each one.

Projects of the odometry lab and how they are run on our robot.

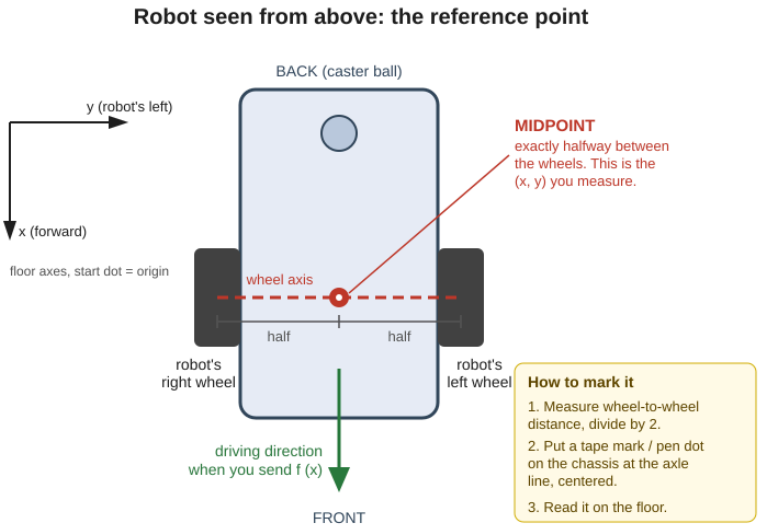
Project	Points	Requirement	Command
P1	0	Functional robot (chassis, servos, LiPo, Raspberry Pi, Nano)	–
P2	1	Rough keyboard moves f, b, r, l (turns about the wheel-axis midpoint)	f, b, r, l
P3	3	Drive 2 m forward and 2 m backward without turning, back to the start within 5 cm, using a P or PI controller	w
P4	0.5	30 runs along a 40 cm line: covariance matrix of the end positions; 30 left and 30 right 90° turns: variances	f40, r90, l90
P5	3.5	Two autonomous laps of a 40 cm square (bow-tie along the diagonals, then the outer edges), end within 5 cm of the origin	x (x1, x2)
Report	1	Report on GitHub	–

All work was done in a single Arduino sketch that grew over the lab (the final version is in the GitHub repository). All parameters can be changed at run time from the Serial Monitor (k, see Appendix A), so most tuning did not need a re-upload. Every move prints one MOVE line with the commanded target, the wheel rotations achieved, the controller error and the duration. These logs are the data source for this report.

2. Hardware, reference point and coordinate frame

- **Drive:** two Parallax Feedback 360 high-speed servos (continuous rotation with a Hall-effect angle sensor, feedback as a 910 Hz PWM signal), and a caster ball at the back.
- **Controller:** Arduino Nano 33 BLE (3.3 V logic). Right servo on pin 3 with feedback on pin 2, left servo on pin 5 with feedback on pin 4.
- **Power:** LiPo battery for the servos (handled according to the LiPo safety sheet).
- **Geometry (measured):** wheel diameter 6.5 cm to 6.6 cm, distance between the wheels 10.5 cm. The *effective* values used by the firmware were calibrated later (Sections 5 and 7).

The robot's position is the midpoint of the wheel axis (Figure 1), which is also the point it rotates about during a turn on the spot. All measurements use this point. The frame is the same for every experiment: x points along the start heading (the direction of f), y points to the robot's left, and angles are positive counter-clockwise.



Top view of the robot. The wheel-axis midpoint is the reference point for all position measurements. The caster ball is at the back.

3. Encoder feedback and odometry model

3.1 Reading the wheel angle

The feedback line carries a PWM signal whose duty cycle encodes the shaft angle. Following the Parallax datasheet (dc_min = 2.9 %, dc_max = 97.1 %), one high and one low pulse are measured with pulseIn(). The sample is accepted only if the cycle time is plausible (1000 µs < t_cycle < 1200 µs), and the angle is

$$\vartheta = 359^\circ - \frac{(dc - 2.9)\cdot 360^\circ}{97.1 - 2.9 + 0.1}.$$

The handout formula has "+1" in the denominator. That is the datasheet's per-mille offset; converted to percent, it is +0.1. With +1, our simulation of the formula showed an angle error of up to 4° over the range, against 0.6° with +0.1.

3.2 Counting rotations, wrap-around and dead zone

Each wheel keeps a running total of rotation in degrees (forward positive). For every new sample, the change $\Delta\vartheta$ is wrapped into $(-180°, 180°]$, which handles the 359° → 0° crossing. Near 0° the sensor has a dead zone in which it can report an angle that is 180° off (lab notes). We therefore:

1. treat any jump larger than 60° between two samples as implausible,
2. correct and count it if it is exactly 180° off (DZ_FIX, switch kz), and
3. otherwise ignore the sample, and re-synchronise after 4 rejected samples in a row.

The filter also stays on while the wheels coast after the servos are detached, because a coasting wheel can cross the dead zone as well.

3.3 From wheel rotation to robot motion

With the effective wheel perimeter P, wheel rotations $\Delta\varphi_R$, $\Delta\varphi_L$ (in degrees) become distances $s_{R,L} = \Delta\varphi_{R,L}\, P/360^{\circ}$. For an effective wheelbase b, the robot moves

$$\Delta s = \frac{s_R + s_L}{2}, \qquad \Delta\theta = \frac{s_R - s_L}{b},$$

and the pose is updated with midpoint integration:

$$x \leftarrow x + \Delta s \cos\left(\theta + \frac{\Delta\theta}{2}\right), \quad y \leftarrow y + \Delta s \sin\left(\theta + \frac{\Delta\theta}{2}\right), \quad \theta \leftarrow \theta + \Delta\theta.$$

Two refinements turned out to matter:

- **Trim-corrected heading on straight moves.** The wheels are not exactly the same size, so on straight moves the right wheel is deliberately driven a fraction ε less than the left (Section 5.1). That intended difference is removed before computing $\Delta\theta$, otherwise every straight side would appear to be a turn of about 1°.
- **Separate wheelbases.** Straight moves, right turns and left turns each use their own effective b (Section 7), because the wheels slip differently in each case.

4. Project 2: Rough movement

Our first sketch moved the servos for a fixed time (500 ms per command) at hand-found offsets from the stop values, with no feedback at all. Five forward commands covered about 46.5 cm with 2.6 cm of drift to the right. Five backward commands covered 53.7 cm with 3.2 cm of drift. Forward and backward were clearly not symmetric, and the distance per command depended on the battery.

We then replaced time-based moves with *distance-based* moves, which already satisfies Project 2: f[cm], b[cm], r[deg], l[deg] drive until the accumulated wheel rotation reaches the target. For a turn on the spot, each wheel has to travel the arc $\pi b\,\alpha/360^{\circ}$ in opposite directions, so the wheel target in degrees is $\alpha\, b/(P/\pi)$. The command parser also accepts several moves on one line (e.g. f40 r90 f40) and a stop command s. After every move the servos are detached to avoid jitter, as the handout recommends.

5. Project 3: Driving straight

5.1 Controller design

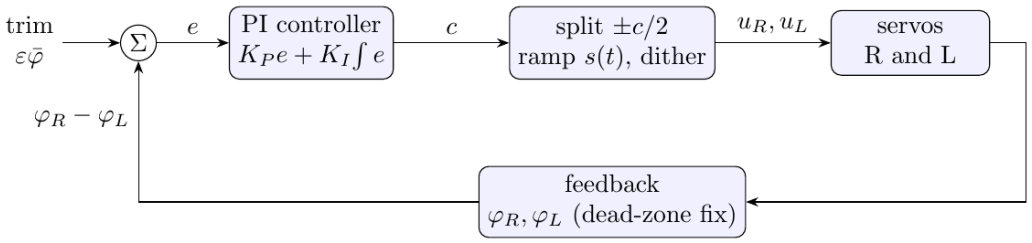
Figure 2 shows the control loop that runs every 20 ms during a move. The controlled quantity is the difference between the two wheel rotations since the start of the move, corrected by a trim term:

$$e = \left(\varphi_R - \varphi_L\right) + \varepsilon\,\varphi^{-}, \qquad \varphi^{-} = \tfrac{1}{2}\left(\varphi_R + \varphi_L\right).$$

Here ε (TRIM_EPS forward, TRIM_EPS_B backward) makes the right wheel turn slightly less than the left, to compensate for the slightly different wheel sizes the encoders cannot see. A PI law gives the correction c in servo units:

$$c = K_P\, e + K_I \int e\, dt, \qquad |c| \le c_{max},$$

which is applied as $u_R = u_{R,0}\, s(t) - c/2$ and $u_L = u_{L,0}\, s(t) + c/2$. In words, the wheel that is ahead is slowed down and the other one is sped up.



Wheel-synchronisation loop (every 20 ms). The same loop is used for straight moves and turns, each with its own K_P. Turns use no trim.

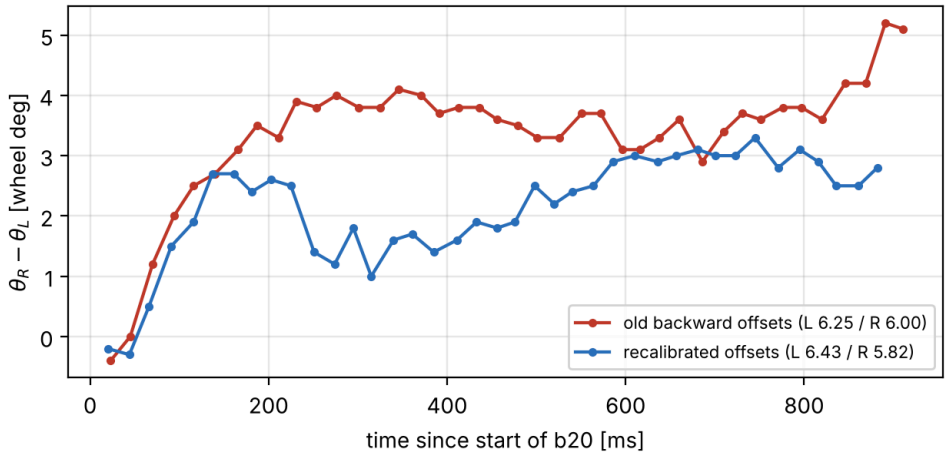
Why PI and not P only. With P control alone, a constant speed mismatch between the servos can only be held by a constant error. Our logs showed a residual of −4.6° to −5.8° of wheel difference forward and +3.4° to +5.4° backward. One servo unit changes the wheel speed by about 8 % (about 23°/s), so the error decays as $\dot{e} \approx -\,23K_P\, e$. A P controller would need a K_P of 0.15 or more to stay below 2°, which starts to cause oscillation. Adding an I term removes the steady error. We used K_P = 0.08 and K_I = 0.03, which settle in about 4–5 s, well within the 14 s of a 2 m move. The I term is protected against wind-up: it does not integrate while the output is saturated in the direction of the error, and its share is limited to 1 servo unit.

Supporting pieces.

- **Fractional speeds (dithering).** One integer step of Servo.write() is about 8 % of the speed, which is far too coarse for a 1–2 % correction. The fractional command is therefore spread over successive control cycles with error diffusion, so the time average matches the fractional value.
- **Ramps.** The speed starts at 90 % and rises to 100 % over 250 ms. In the last 150° of wheel rotation it falls to 80 %. A lower limit (MIN_OFF = 4) keeps both wheels above their stall point.
- **Coast allowance.** The servos are detached 26° of wheel rotation before the target (turns: 20°), and the wheels coast to the target. The value was found by setting it to 0 and averaging the overshoot.
- **End-of-move correction (removed).** We tried small corrective pulses after a move to bring e back to zero. Once the I term was in place it added almost nothing (simulated heading 0.28° with it, 0.23° without), so we removed it.

5.2 Calibration

1. **Servo offsets.** The stop values are 93 (left) and 94 (right). The base offsets are 6.8 (L) and 8.0 (R) forward, and 6.48 (L) and 5.77 (R) backward. The backward values were found from controller traces (below).
2. **Wheel perimeter.** Six 2 m runs measured with a tape gave a ruler distance of 201.25 cm on average, against 200.09 cm from the encoders. This ratio of 1.0058 was the same forward and backward, and changed P from π·6.6 = 20.73 cm to 20.85 cm. In a final check, the forward leg of w still ended about 1.5 cm short, so we set P = 20.69 cm. All wheelbases were scaled by the same factor (0.9925), so the turns stayed unchanged, because the turn target depends only on b/P.
3. **Trim.** Forward ε = 0.0055, later 0.0048. Backward ε_B = 0.0020. These were tuned by measuring the sideways offset at the end of 2 m runs.
4. **Backward offsets from traces.** Every b20 started with a small jerk to the right. The trace (Figure 3) showed that the right wheel pulled about 4° ahead in the first 250 ms, and that the controller then held a constant correction of about 0.35 units. So the base offsets did not match the real wheel speeds. Setting the base offsets to the values the controller was actually applying (6.25/6.00 → 6.43/5.82, later 6.48/5.77) halved the start error. Increasing the ramp length instead would not have helped, because the backward offsets are only about 2 units above the stall point.



Wheel difference during a 20 cm backward move, before and after recalibrating the backward base offsets. The start error falls from about 4° to about 2.7°, and the controller pulls it back towards its set point.

5.3 Results

The table lists four complete w runs logged with the tuned PI controller. The large diff values are the intended trim (about 19° forward and 10° backward over 2 m); the controller error err stays within ±3° of wheel rotation, which is about ±1° of heading.

Encoder results of four w runs (2 m forward, 1.5 s pause, 2 m backward).

Run	err fwd [°]	err back [°]	net s_R [cm]	net s_L [cm]	net rot. [°]
1	0.9	−1.7	0.1	0.4	−2.0
2	−0.7	2.8	−0.4	0.4	−3.9
3	2.0	−1.2	−0.4	−0.1	−1.7
4	0.9	1.3	−0.3	0.2	−2.9

The net rotation printed by the firmware does not take the trim into account and should not be read as a real heading. On the floor, the robot came back to within about 1 cm to 2 cm of the start line. On average, the forward leg ended 1.5 cm short of the 2 m mark before the perimeter recalibration and on the mark afterwards, and the backward leg overshot the start by about 0.8 cm. Run-to-run scatter was about ±1° to 2° of heading. Most of it comes from a small twist at the start of each leg. The controller then corrects the heading, but the robot keeps the sideways offset, which the encoders cannot detect.

6. Project 4: covariance matrix and turn variances

6.1 Straight 40 cm runs

The robot was placed on a taped L-shaped stop (same position and heading every time) and sent f40 30 times. After each run the end position of the wheel-axis midpoint was measured to 0.1 cm (table below). Following the handout, the covariance matrix is

$$\Sigma = \frac{1}{N}\sum_{i=1}^{N}\left(x_i - \bar{x}\, y_i - \bar{y}\right)\left(x_i - \bar{x}\, y_i - \bar{y}\right), \qquad N = 30.$$

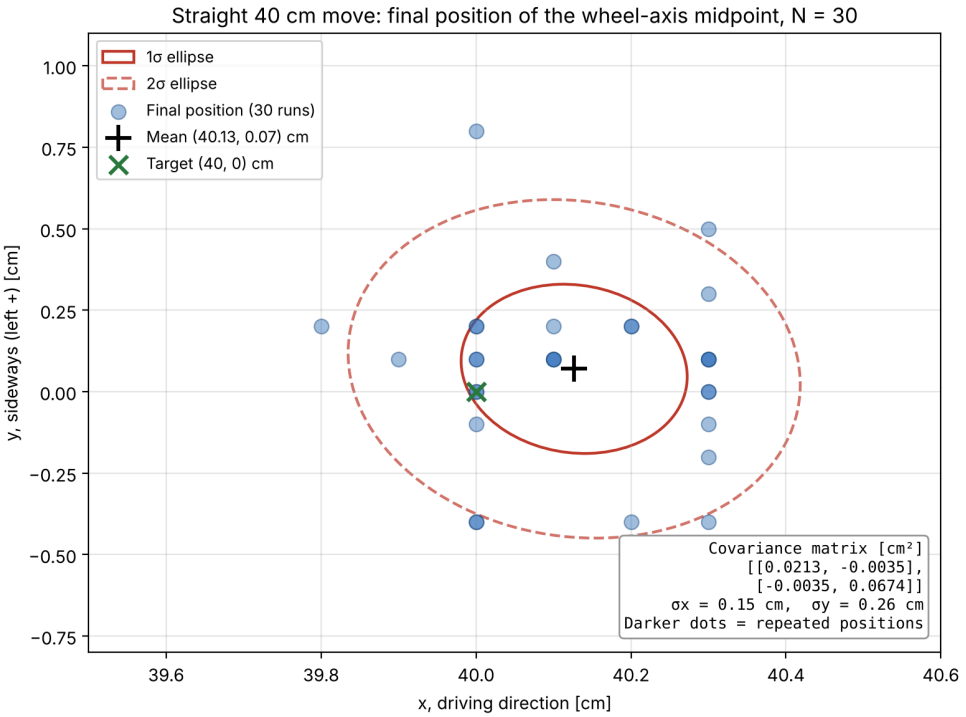
End positions of the 30 runs along the 40 cm line (x along the line, y to the left), in cm.

Run	x	y	Run	x	y	Run	x	y
1	40.2	−0.4	11	40.0	−0.4	21	40.0	0.1
2	40.3	0.3	12	39.8	0.2	22	40.0	−0.4
3	40.3	−0.2	13	40.1	0.2	23	40.2	0.2
4	40.3	0.1	14	40.3	0.5	24	40.0	0.0
5	40.3	0.1	15	40.0	0.0	25	40.1	0.1
6	40.3	0.0	16	40.0	0.1	26	40.3	0.0
7	40.3	−0.1	17	39.9	0.1	27	40.3	−0.4
8	40.0	0.2	18	40.0	0.8	28	40.0	0.2
9	40.3	0.1	19	40.0	−0.1	29	40.1	0.1
10	40.2	0.2	20	40.1	0.4	30	40.1	0.1

Result:

$$\bar{x} = 40.13\ cm, \quad \bar{y} = 0.07\ cm, \qquad \Sigma = (0.0213\ \ -0.0035\ \ -0.0035\ 0.0674)\ cm^2,$$

$$\Sigma = \begin{pmatrix} 0.0213 & -0.0035 \\ -0.0035 & 0.0674 \end{pmatrix}\ \mathrm{cm}^2$$



End positions of the 30 runs with mean, target and 1σ/2σ ellipses. Darker dots are repeated positions, because the readings have a resolution of 0.1 cm.

Interpretation.

- The robot overshoots by only 1.3 mm on average, which is the systematic part. The random scatter is in the millimetre range, against the “few centimetres” expected in the handout and the 5 cm tolerance of Project 3.
- The sideways scatter is about twice as large as the scatter along the line. That is expected, because heading errors become sideways errors. The covariance is almost zero, so the ellipse is aligned with the axes: the distance error and the sideways error are independent.
- One consequence for the next steps: straight driving was good enough, and turns became the critical part of Project 5.

6.2 Turn variances (30 right and 30 left 90° turns)

The robot was sent r90 thirty times and l90 thirty times. After each turn the real angle was measured on the floor (table below). As for the position, we use the 1/N variance of the readings; the mean minus 90° is the bias of the turn.

Measured angle in degrees after 30 right and 30 left 90° turns.

Turn	Right 90°	Left 90°	Turn	Right 90°	Left 90°
1	90.00	91.50	16	90.25	92.00
2	91.00	90.00	17	90.00	91.50
3	90.50	92.50	18	94.00	92.50
4	90.00	92.50	19	94.00	90.50
5	92.00	93.50	20	91.00	92.50
6	89.50	94.00	21	94.00	93.00
7	94.00	92.00	22	92.00	91.00
8	91.00	89.50	23	93.00	90.50
9	92.50	92.00	24	92.50	94.00
10	91.50	91.50	25	92.50	93.00
11	91.50	92.00	26	92.50	91.50
12	92.00	90.50	27	92.50	91.00
13	91.00	90.50	28	92.00	94.50
14	90.50	92.00	29	90.50	92.50
15	94.00	91.50	30	93.00	92.00

Statistics of the 90° turns.

	n	Mean [°]	Bias [°]	Variance, 1/N [deg²]	Variance, 1/(N−1) [deg²]	Std. dev. [°]	Min – max [°]
Right 90°	30	91.83	+1.83	1.83	1.89	1.35	89.5 – 94.0
Left 90°	30	91.92	+1.92	1.40	1.45	1.18	89.5 – 94.5

Interpretation.

- Right and left turns behave alike. Both overshoot by about 1.8° to 1.9° on average, and their standard deviations are 1.35° and 1.18°. The difference between the two is within the normal sampling noise of 30 readings. This supports a common turn model with separate wheelbases that differ only a little.
- The mean offset is the systematic part and could be removed by scaling the turn wheelbase. The standard deviation is the random part that no calibration can remove. Part of it is reading error of the measurement (about ±0.5°).
- During calibration we also measured smaller samples of 135° turns and one of 90° turns (table below). They show the same order of magnitude: a standard deviation of about 1° for 90° turns, and 1.4° to 1.8° for 135° turns at the original turn speed.

Turn measurements taken during calibration (protractor sheet, n = 5 each).

Turn	Readings [°]	Mean [°]	Variance [deg²]	SD [°]
r90 (b = 10.5)	92.5, 91.5, 90.0, 91.0, 91.1	91.22	0.65	0.90
r135 (b = 10.59)	130, 130, 131, 128, 133	130.4	2.64	1.82
l135 (b = 10.59)	130.5, 129.5, 130, 130, 127	129.4	1.54	1.39

The means of these small samples were used to calibrate the effective wheelbase, $b_{new} = b_{old} \cdot \alpha_{encoder}/\alpha_{real}$ (Section 7).

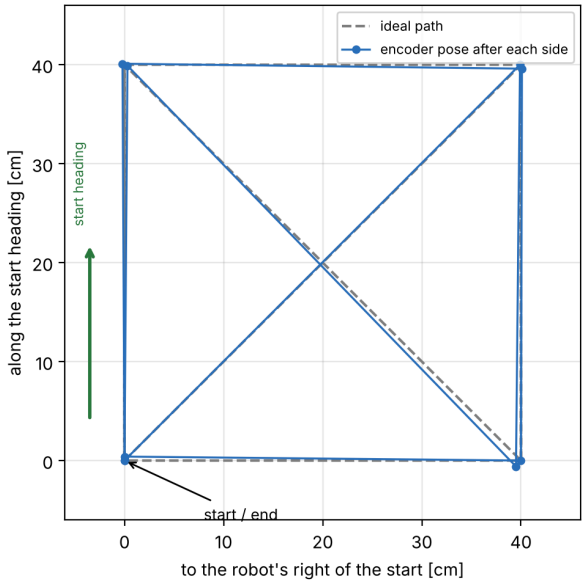
7. Project 5: flexible odometry

7.1 Path

Both laps use the 40 cm square of the covariance experiment. In our frame (start corner at the origin, facing +x), the waypoints are:

Lap	Corners	Turns
1 (bow-tie)	(40,0) → (0,−40) → (40,−40) → (0,0)	R135, L135, L135
2 (square)	(40,0) → (40,−40) → (0,−40) → (0,0)	R135, R90, R90, R90
end	back to the start heading	R90

The sides are 40 cm and the diagonals $40\sqrt{2}$ = 56.6 cm. Figure 5 shows the ideal path together with the odometric pose of one of our best runs.



Ideal Project 5 path (dashed) and the encoder pose after every side in one full x run with the final settings (both compensators on, turn speed 0.75). The encoders put the end point 0.5 cm from the origin.

7.2 The x command

x runs both laps without any interaction. x1 and x2 run only one lap, which we used for calibration. All three end with a turn back to the start heading. The robot pauses 600 ms after every side and every turn so that it is completely still before the next move. For every segment the firmware:

1. updates the pose (x, y, θ) from the encoders (Section 3),
2. **aims the turn at the next corner** (turn compensation, kj): the turn is computed as atan2(y_c − y, x_c − x) − θ instead of the nominal angle, limited to ±8° around it. Errors of earlier turns and sides are therefore corrected at the next corner,
3. **drives the remaining distance** (side-length compensation, km): the length of the side is the projection of the vector to the corner onto the current heading, limited to ±3 cm around the nominal length,
4. **learns where to stop each turn**: 135° and 90° turns have their own coast value. After every turn, half of the measured stopping error is fed back into that value for the next turn of the same size.

7.3 Turn calibration

Turns turned out to be the main error source, for three reasons.

- **Stopping point.** With one coast value for all turns, 135° turns stopped on average 5.2° of wheel rotation short, but 90° turns only 1.3°. Separate, learned coast values per turn size fixed this.
- **Effective wheelbase.** The encoders reported the turns as correct while the floor showed 135° turns about 4° to 5° short. Comparing the measured angle with the encoder angle (table below) showed that right and left turns need different values. One compromise value would have made both errors push the heading the same way, adding up to about 3° per bow-tie lap.
- **Turn speed.** Faster turns stopped less repeatably. A turn-only speed factor of 0.75 (straight moves unchanged) halved the scatter of the stopping point (Figure 6).

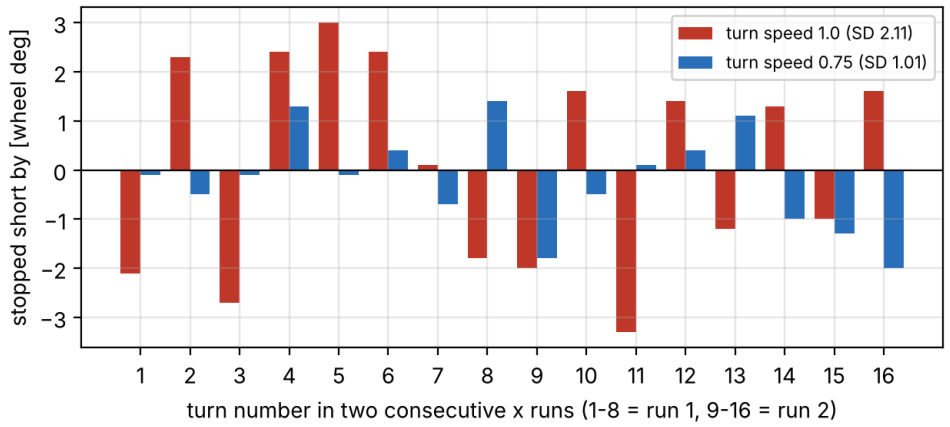
Wheelbase calibration from five right and five left 135° turns with b = 10.59 cm.

	Encoder angle (mean)	Real angle (mean)	Effective wheelbase
Right turns	131.8°	130.4°	10.59 × 131.8/130.4 = 10.71 cm
Left turns	129.4°	129.4°	10.59 cm

The final values were then fine-tuned from the end heading of complete laps. x2 contains only right turns and isolates b_R. x1 then sets b_L:

$$b_{R,new} = b_R\left(1 + \frac{\theta_2}{360^\circ}\right), \qquad b_{L,new} = b_L\left(1 - \frac{\theta_1}{270^\circ}\right),$$

where θ_1, θ_2 is the mean real end heading (positive = turned left) of five runs. The factors come from a sensitivity analysis of the path with the compensators on (Section 8.2). This resulted in b_R = 10.58 cm and b_L = 10.74 cm, which became 10.50 and 10.66 cm after the perimeter rescaling.



Stopping error of all 16 turns in two consecutive full x runs, at the original turn speed and with a turn speed factor of 0.75.

7.4 Approaches we tried and dropped

- **Heading fix pulses after each turn** (kr): short slow pulses until the encoder heading was within 1°. The robot wiggled visibly, so we replaced this with the coast learning above.
- **Steering onto the ideal line during the sides** (kh, look-ahead 15 cm): this worked in simulation, but it showed a gain of only about 1 mm, so we kept the simpler version.
- **Slower coast learning** (0.15 instead of 0.5) and a **stronger turn controller** (K_P,T = 0.08 instead of 0.05): both made the floor results worse, so both were reverted.

7.5 Results

The table lists representative runs in the order they were made. “Encoder” is the distance from the origin according to the firmware, and “floor” is what we measured.

Selected Project 5 runs. Comps = turn and side-length compensation (kj, km).

Mode	Setting	Encoder [cm]	Floor / remark
x1	one wheelbase 10.38, learned coast	1.5	last turn 4° to 5° off → wheelbase recalibration
x1	b = 10.71, comps off (3 runs)	1.3, 2.9, 1.6	2.1 cm right, 2.0 cm ahead of the origin
x1	b_R/b_L separate, comps off	1.5, 1.2	ends at the origin, tilted slightly to the right
x1	K_P,T = 0.08 (4 runs)	0.4 – 3.6	visibly worse → reverted
x	comps off, fresh battery	2.0	end heading −6.7° (no correction between laps)
x	comps on, b_R = 10.58, b_L = 10.74	0.8	**1.5 cm ahead, centred on the line**
x	comps on, turn speed 0.75 (2 runs)	0.5, 1.1	best runs, visually on the origin

All runs ended well inside the 5 cm circle needed for full points. Two practical observations:

- **Battery state** changes the speed by 6–7 % between a fresh and a partly discharged LiPo. That shifts the coast values. We therefore always start with one warm-up run and compare the side times in the log (about 2.9 s per 40 cm side) with the reference.
- **A careful start matters.** A start heading error of 1° alone moves the end point by up to about 2 cm. We used a taped L-shape for the start pose.

8. Error analysis

8.1 Sensitivity to calibration errors

The table shows how one systematic calibration error moves the real end point when both compensators are on. Two conclusions follow. The end heading is the most sensitive and easiest quantity to measure, which is why it is used to calibrate b_R and b_L. And the wheel perimeter has no effect on a closed path, because a scale error cancels out over the loop.

Effect of one systematic error on the real end of the path (compensators on).

Error	End position of x	End heading x	End heading x2	End heading x1
Right turns 1 % too large	1.3 cm ahead, 2.1 cm left	6.3° R	3.6° R	2.7° R
Left turns 1 % too large	1.9 cm behind, 1.0 cm right	2.7° L	–	2.7° L
Distances 1 % too long	no effect	–	–	–
Sides curve 0.25° per 40 cm	0.8 cm ahead, 0.8 cm left	2.2° R	1.0° R	1.2° R

9. Conclusion and lessons learned

- **Feedback first.** Moving by measured wheel rotation instead of time, with correct handling of the wrap-around and the dead zone, removed most of the open-loop errors at once.
- **PI instead of P.** The I term removed the steady wheel-difference error of about 5°. Setting the base offsets to the speeds the controller actually needs reduced the start transient.
- **Straight driving is very repeatable** (σ_x = 1.5 mm, σ_y = 2.6 mm over 40 cm). The difficult part of the lab was turning: 90° turns scatter by about 1.2° to 1.35° (standard deviation) around a mean that is about 1.8° too large.
- **Calibrate against the floor, not against the encoders.** Several times the logs looked perfect while the robot was off. Effective wheelbases per turn direction, learned stopping points and slower turns brought the real results close to the encoder results.
- **Change one parameter at a time.** Changing two settings at once (e.g. K_P,T and b_R) made it impossible to tell which change helped.

Final result: Project 3 returns within 1 cm to 2 cm, Project 4 shows millimetre-level scatter for straight runs and about 1.2° to 1.35° scatter for 90° turns, and Project 5 ends within 0.5 cm to 1.5 cm of the origin after both laps. All three are well within the required tolerances.

Appendix A: Serial Monitor commands

9600 baud, line ending “Newline”. Several commands may be typed on one line.

Command	Function
f[cm], b[cm]	forward / backward (default 20 cm)
r[deg], l[deg]	turn right / left on the spot (default 45°)
w	Project 3: 200 cm forward, 1.5 s pause, 200 cm back
q[laps]	square of 40 cm with turn error carried into the next turn
x, x1, x2	Project 5: both laps / bow-tie only / square only, each with the final turn
s	stop the running move (also aborts x and q in a pause)
z, p, h, v	zero counters / print wheel info / print parameters / trace on–off
k	set parameter to : f b t Kp forward/backward/turn, i Ki, c max. correction, p perimeter, w wheelbase (straight), R L wheelbase right/left turns, o a coast straight/turn, x X trim forward/backward, u s d e ramps, y speed shift, T turn speed, z dead-zone fix, q n square side/pause, j m compensators, A B x coast 135°/90°

Appendix B: Final parameters

Parameter	Value	Parameter	Value
Stop value L / R	93 / 94	K_P forward / backward / turn	0.08 / 0.08 / 0.05
Offsets forward L / R	6.8 / 8.0	K_I, I limit, c_max	0.03, 1.0, 2.0
Offsets backward L / R	6.48 / 5.77	Trim forward / backward	0.0048 / 0.0020
Minimum offset	4	Coast straight / turn	26° / 20°
Wheel perimeter	20.69 cm	Turn speed factor	0.75
Wheelbase straight	10.30 cm	Ramp up / start	250 ms / 0.9
Wheelbase right turns	10.50 cm	Ramp down / end	150° / 0.8
Wheelbase left turns	10.66 cm	x coast 135° / 90° (start)	11.1 / 15.1
Square side / pause	40 cm / 600 ms	Compensators kj, km	on, on

Photos:

